

CHAPTER 10

HARDWARE TESTING AND DESIGN FOR TESTABILITY

This chapter introduces digital system testing and design methods that make the systems easier to test. We have already discussed the use of testing during the design process. We have written VHDL test benches to verify that the overall design and algorithms used are correct. We have used simulation at the logic level to verify that a design is logically correct and that it meets specifications. After the design of an IC is completed, additional testing can be done by simulating it at the circuit level to verify that the design has been correctly implemented and that the timing is correct.

When a digital system is manufactured, further testing is required to verify that it functions correctly. When multiple copies of an IC are manufactured, each copy must be tested to verify that it is free from manufacturing defects. This testing process can become very expensive and time consuming. With today's complex ICs, the cost of testing is a major component of the manufacturing cost. Therefore, it is very important to develop efficient methods of testing digital systems and to design the systems so that they are easy to test.

In this chapter, we first discuss methods of testing combinational logic for the basic types of faults that can occur. Then we describe methods for determining test sequences for sequential logic. One of the problems encountered is that normally we have access only to the inputs and outputs of the circuit being tested and not to the internal state. To remedy this problem, internal test points may be brought out to additional pins on the IC. To reduce the number of test pins required, we introduce the concept of scan design, in which the state of the system can be stored in a shift register and shifted out serially. Finally, we discuss the concept of built-in self-test. By adding more components to the IC, we can generate test sequences and verify the response to these sequences internally without the need for expensive external testing.

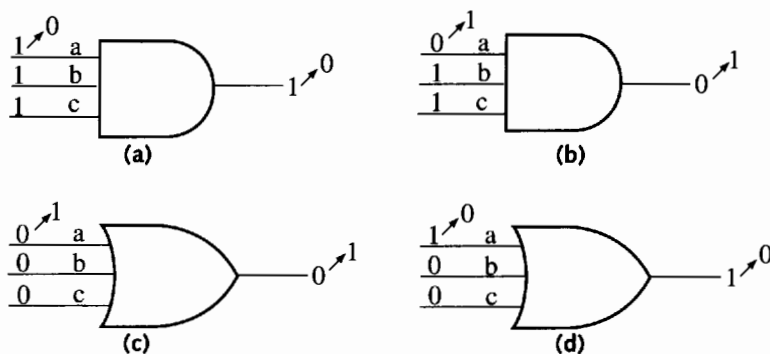
10.1 TESTING COMBINATIONAL LOGIC

Two common types of faults are short circuits and open circuits. If the input to a gate is shorted to ground, the input acts as if it is stuck at a logic 0. If the input to a gate is shorted to a positive power supply voltage, the gate input acts as if it is stuck at a logic 1. If the input to a gate is an open circuit, the input may act as if it is stuck at 0 or stuck at 1, depending on the type of logic being used. Thus, it is common practice to model faults in logic circuits as stuck-at-1 (s-a-1) or stuck-at-0 (s-a-0) faults. To test a gate input for s-a-0,

the gate input must be 1 so a change to 0 can be detected. Similarly, to test a gate input for s-a-1, the normal gate input must be 0 so a change to 1 can be detected.

We can test an AND gate for s-a-0 faults by applying 1s to all inputs, as shown in Figure 10-1(a). The normal gate output is then 1, but if any input is s-a-0, the output becomes 0. The notation $1 \rightarrow 0$ on the gate input a means that the normal value of a is 1, but the value has changed to 0 because of the s-a-0 fault. The notation $1 \rightarrow 0$ at the gate output indicates that this change has propagated to the gate output. We can test an AND gate input for s-a-1 by applying 0 to the input being tested and 1s to the other inputs, as shown in Figure 10-1(b). The normal gate output then is 0, but if the input being tested is s-a-1, the output becomes 1. To test OR gate inputs for s-a-1, we apply 0s to all inputs, and if any input is s-a-1, the output will change to 1 (Figure 10-1(c)). To test an OR gate input for s-a-0, we apply a 1 to the input under test and 0s to the other inputs. If the input under test is s-a-0, the output will change to 0 (Figure 10-1(d)). In the process of testing the inputs to a gate for s-a-0 and s-a-1, we also can detect s-a-0 and s-a-1 faults at the gate output.

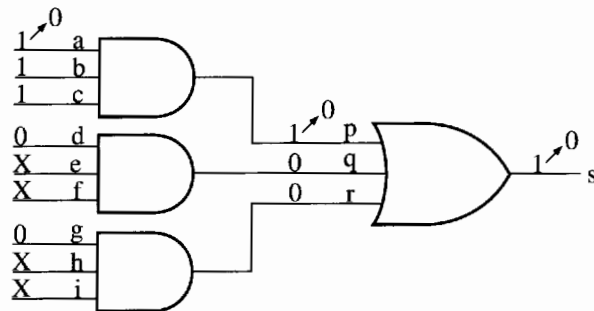
Figure 10-1 Testing AND and OR Gates for Stuck-at Faults



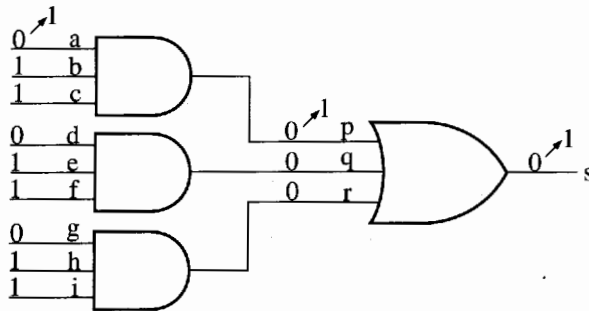
The two-level AND-OR network of Figure 10-2 has nine inputs and one output. We assume that the OR gate inputs (p , q , and r) are not accessible, so the gates cannot be tested individually. One approach to testing the network would be to apply all $2^9 = 512$ different input combinations and observe the output. A more efficient approach is based on testing for all s-a-0 and s-a-1 faults, as shown in Table 10-1. To test the abc AND gate inputs for s-a-0, we must apply 1s to a , b , and c , as shown in Figure 10-2(a). Then, if any gate input is s-a-0, the gate output (p) will become 0. In order to transmit the change to the OR gate output, the other OR gate inputs must be 0. To achieve this, we can set $d = 0$ and $g = 0$ (e , f , h , and i are then don't cares). This test vector will detect $p0$ (p stuck-at-0) as well as $a0$, $b0$, and $c0$. In a similar manner, we can test for $d0$, $e0$, $f0$, and $g0$ by setting $d = e = f = 1$ and $a = g = 0$. A third test with $g = h = i = 1$ and $a = d = 0$ will test the remaining s-a-0 faults. To test a for s-a-1 ($a1$), we must set $a = 0$ and $b = c = 1$, as shown in Figure 10-2(b). Then, if a is s-a-1, p will become 1. In order to transmit this change to the output, we must have $q = r = 0$, as before. However, if we set $d = g = 0$ and $e = f = h = i = 1$, we can test for $d1$ and $g1$ at the same time as $a1$. This same test vector also tests for $p1$, $q1$, and $r1$. As shown in the table, we can test for $b1$, $e1$, and $h1$ with a single test vector and test similarly for $c1$, $f1$, and $i1$. Thus we can test all s-a-0 and s-a-1 faults with only six tests, whereas the brute-

force approach would require 512 tests. When we apply the six tests, we can determine whether or not a fault is present, but we cannot determine the exact location of the fault. In the preceding analysis, we have assumed that only one fault occurs at a time. In many cases the presence of multiple faults will also be detected.

Figure 10-2 Testing an AND-OR Network



(a) Stuck-at-0 test



(b) Stuck-at-1 test

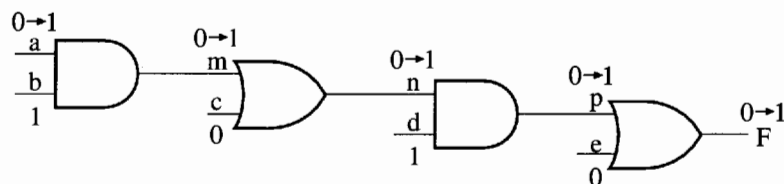
Table 10-1 Test Vectors for Figure 10-2

a	b	c	d	e	f	g	h	i	Faults Tested
1	1	1	0	X	X	0	X	X	a0, b0, c0, p0
0	X	X	1	1	1	0	X	X	d0, e0, f0, q0
0	X	X	0	X	X	1	1	1	g0, h0, i0, r0
0	1	1	0	1	1	0	1	1	a1, d1, g1, p1, q1, r1
1	0	1	1	0	1	1	0	1	b1, e1, h1, p1, q1, r1
1	1	0	1	1	0	1	1	0	c1, f1, i1, p1, q1, r1

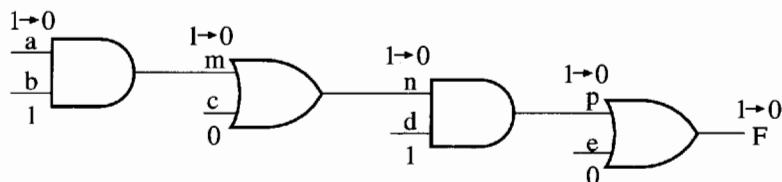
Testing multilevel networks is considerably more complex than testing two-level networks. In order to test for an internal fault in a network, we must choose a set of inputs that will excite that fault and then propagate the effect of that fault to the network output. In Figure 10-3, *a*, *b*, *c*, *d*, and *e* are network inputs. If we want to test for gate input *n*

s-a-1, n must be 0. This can be achieved if we make $c = 0$, $a = 0$, and $b = 1$, as shown. In order to propagate the fault n s-a-1 to the output F , we must make $d = 1$ and $e = 0$. With this set of inputs, if a , m , n , or p is s-a-1, the output F will have the incorrect value and the fault can be detected. Furthermore, if we change a to 1 and gate input a , m , n , or p is s-a-0, the output F will change from 1 to 0. We say that the path through a , m , n , and p has been sensitized, since any fault along that path can be detected. The method of path sensitization allows us to test for a number of different stuck-at faults using one set of network inputs.

Figure 10-3 Fault Detection Using Path Sensitization



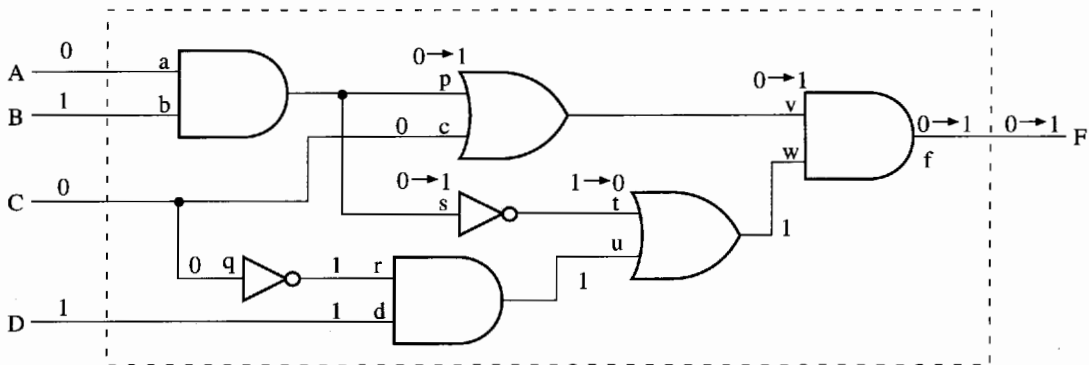
(a) s-a-1 tests



(b) s-a-0 tests

Next, we try to determine a minimum set of test vectors to test the network of Figure 10-4 for all single stuck-at-1 and stuck-at-0 faults. We assume that we can apply inputs to A , B , C , and D and observe the output F and that the internal gate inputs and outputs cannot be accessed. Suppose that we want to test input p for s-a-1. In order to do this, we must choose inputs A , B , C , and D such that $p = 0$, and if p is s-a-1, we must propagate this fault to the output F so it can be observed. In order to propagate the fault, we must make $c = 0$ and $w = 1$. We can make $w = 1$ by making $t = 1$ or $u = 1$. To make $u = 1$, we must have both D and $r = 1$. Fortunately, our choice of $C = 0$ makes $r = 1$. To make $p = 0$, we choose $A = 0$. By choosing $B = 1$, we can sensitize the path A - a - p - v - f - F so that the set of inputs $ABCD = 0101$ will test for faults $a1$, $p1$, $v1$, and $f1$. This set of inputs also tests for c s-a-1. We assume that c s-a-1 is a fault internal to the gate, so it is still possible to have $q = 0$ and $r = 1$ if c s-a-1 occurs.

Figure 10-4 Example Network for Stuck-at Fault Testing



To test for s-a-0 inputs along the path $A-a-p-v-f$, we can use the inputs $ABCD = 1101$. In addition to testing for faults $a0$, $p0$, $v0$, and $f0$, this input vector also tests the following faults: $b0$, $w0$, $u0$, $r0$, $q1$, and $d0$. To determine tests for the remaining stuck-at faults, we can select an untested fault, determine the required $ABCD$ inputs, and then determine the additional faults that are tested. Then we can repeat this procedure until tests are found for all of the faults. Table 10-2 lists a set five test vectors that will test for all single stuck-at faults in Figure 10-4.

Table 10-2 Tests for Stuck-at Faults in Figure 10-4

				Normal Gate Inputs												Faults Tested
A	B	C	D	a	b	p	c	q	r	d	s	t	u	v	w	
0	1	0	1	0	1	0	0	0	1	1	0	1	1	0	1	a1 p1 c1 v1 f1
1	1	0	1	1	1	0	0	1	1	1	0	1	1	1	1	a0 b0 p0 q1 r0 d0 u0 v0 w0 f0
1	0	1	1	1	0	0	1	1	0	1	0	1	0	1	1	b1 c0 s1 t0 v0 w0 f0
1	1	0	0	1	1	1	0	0	1	0	1	0	0	1	0	a0 b0 d1 s0 t1 u1 w1 f1
1	1	1	1	1	1	1	1	1	0	1	1	0	0	1	0	a0 b0 q0 r1 s0 t1 u1 w1 f1

In addition to stuck-at faults, other types of faults, such as bridging faults, may occur. A bridging fault occurs when two unconnected signal lines are shorted together. For a large combinational network, finding a minimum set of test vectors that will test for all possible faults is very difficult and time consuming. For networks that contain redundant gates, testing for some of the faults may be impossible. Even if a comprehensive set of test vectors can be found, applying all of the vectors may take too much time and cost too much. For these reasons, it is common practice to use a relatively small set of test vectors that will test most of the faults. In general, determining such a set of vectors is a difficult and computationally intensive problem. Many algorithms and corresponding computer programs have been developed to generate such sets of test vectors. Computer programs have also been developed to simulate faulty networks. Such programs allow the user to determine what percentage of possible faults are tested by a given set of input vectors.

10.2 TESTING SEQUENTIAL LOGIC

Testing sequential logic is generally much more difficult than testing combinational logic, because we must use sequences of inputs for testing. If we can observe only the input and output sequences and not the state of the flip-flops in a sequential network, a very large number of test sequences may be required. Basically, the problem is to determine if the network under test is equivalent to a correctly functioning network. We will assume that the sequential network being tested has a reset input so we can reset it to a known initial state. If we attempted to test the network using the brute-force approach, we would reset the network to the initial state, apply a test sequence, and observe the output sequence. If the output sequence was correct, then we would repeat the test for another sequence. In theory, according to the definition of state equivalence, we would have to try all possible input sequences, which is an infinite number, to verify that the initial state of the network under test is equivalent to the initial state of the correct network. In practice, if we know that the network has N or fewer states, then we have to apply only input sequences of length less than or equal to $2N - 1$.

As an example, consider a sequential network that has five inputs, one output, and four states. If we used the brute-force approach, we would have to apply all input sequences of length seven or less. At time = 1, we could apply any one of the 2^5 input combinations, and similarly at times 2 through 7. Thus, the total number of test sequences required is

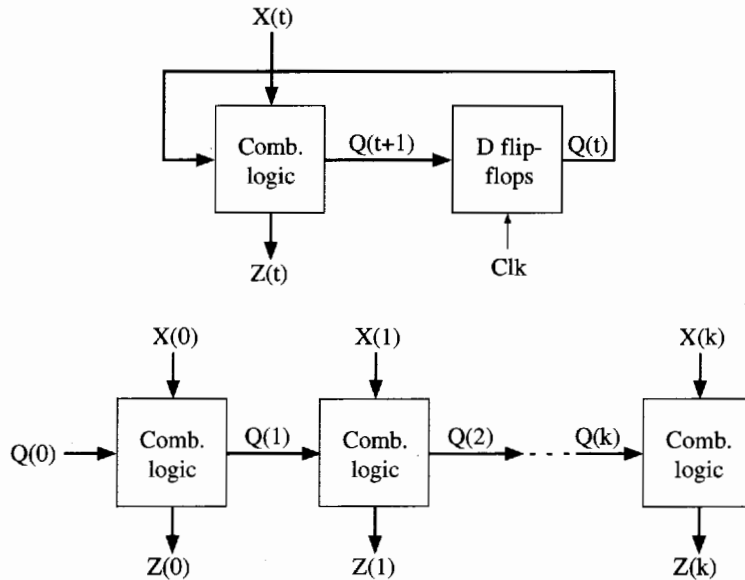
$$(2^5)(2^5)(2^5)(2^5)(2^5)(2^5)(2^5) = 2^{35}$$

Even in this simple example, the number of test sequences would be prohibitive.

Since the brute-force approach is totally impractical, the question then arises, Can we derive a relatively small set of test sequences that will adequately test the network? One way to derive test sequences for a sequential network is to convert it to an iterative network. Since the iterative network is a combinational network, we could then derive test vectors for the iterative network using one of the standard methods for combinational networks.

Figure 10-5 shows a standard Mealy sequential network and the corresponding iterative network. In these figures, X , Z , and Q can either be single variables or vectors. The iterative network has k identical copies of the Comb. network used in the sequential network, where k is the length of the sequence used to test the sequential network. For the sequential network, $X(t)$ represents a sequence of inputs in time. In the iterative network, $X(0) X(1) \dots X(k)$ represents the same sequence in space. Each cell of the iterative network computes $Z(t)$ and $Q(t+1)$ in terms of $Q(t)$ and $X(t)$. The leftmost cell computes the values for $t=0$, the next cell for $t=1$, etc. After the test vectors have been derived for the iterative network, these vectors then become the input sequences used to test the original sequential network. The number of cells in the iterative network depends on the length of the sequences required to test the sequential network.

Figure 10-5 Sequential and Iterative Networks



Derivation of a small set of test sequences that will adequately test a sequential network is generally difficult to do. Consider the state graph shown in Figure 10-6 and the corresponding state table (Table 10-3). We assume that we can reset the network to state S_0 . It is certainly necessary that the test sequence cause the network to go through all possible state transitions, but this is not an adequate test. For example, the input sequence

$$X = 010110011$$

traverses all the arcs connecting the states and produces the output sequence

$$Z = 001011110$$

If we replace the arc from S_3 to S_0 with a self-loop, as shown by the dashed line, the output sequence will be the same, but the new sequential machine is not equivalent to the old one.

Figure 10-6 State Graph for Test Example

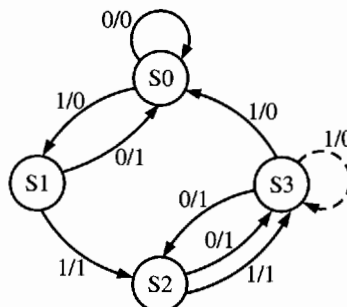


Table 10-3 State Table for Figure 10-6

Q1Q2	State	Next State		Output	
		X = 0	1	X = 0	1
00	S0	S0	S1	0	0
10	S1	S0	S2	1	1
01	S2	S3	S3	1	1
11	S3	S2	S0	1	0

A state graph in which every state can be reached from every other state is referred to as *strongly connected*. A general test strategy for a sequential network with a strongly connected state graph and no equivalent states is to first find an input sequence that will distinguish each state from the other states. Such an input sequence is referred to as a distinguishing sequence. Given a distinguishing sequence, each entry in the state table can be verified. For the example of Figure 10-6, one distinguishing sequence is 11. If we start in S0, the input sequence 11 gives the output sequence 01; for S1 the output is 11; for S2, 10; and for S3, 00. Thus, we can distinguish the four states by using the input sequence 11. We can then verify every entry in the state table using the following sequences, where *R* means reset to state S0:

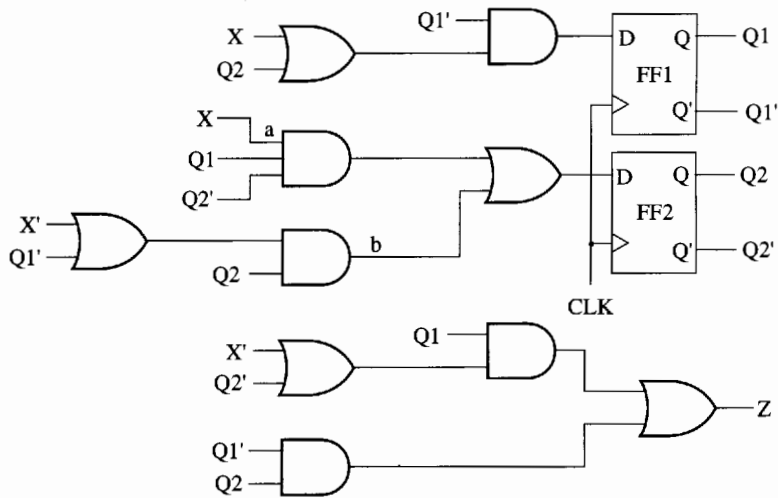
Input	Output	Transition Verified
R 0 1 1	0 0 1	(S0 to S0)
R 1 1 1	0 1 1	(S0 to S1)
R 1 0 1 1	0 1 0 1	(S1 to S0)
R 1 1 1 1	0 1 1 0	(S1 to S2)
R 1 1 0 1 1	0 1 1 0 0	(S2 to S3)
R 1 1 1 1 1	0 1 1 0 0	(S2 to S3)
R 1 1 0 0 1 1	0 1 1 1 1 0	(S3 to S2)
R 1 1 0 1 1 1	0 1 1 0 1 0	(S3 to S0)

Another approach to deriving test sequences is based on testing for stuck-at faults. Figure 10-7 shows the realization of Figure 10-6 using the following state assignment: S0, 00; S1, 10; S2, 01; S3, 11. If we want to test for a s-a-1, we must first excite the fault by going to state S3, in which Q1Q2 = 10 and then setting X = 0. In normal operation, the next state will be S0. However, if *a* is s-a-1, then next state is Q1Q2 = 01, which is S2. This test sequence can then be constructed as follows:

- To go to S1: reset followed by X = 1.
- To test a s-a-1: X = 0.
- To distinguish the state that is reached: X = 11.

The final sequence is R1011. The normal output is 0101, and the faulty output is 0110.

Figure 10-7 Realization of Figure 10-6



We have shown some simple examples that illustrate some of the methods used to derive test sequences for sequential networks. As the number of inputs and states in the network increases, the number and length of the required test sequence increases rapidly, and the derivation of these test sequences becomes much more difficult. This, in turn, means that the time and expense required to test the networks increases rapidly with the number of inputs and states.

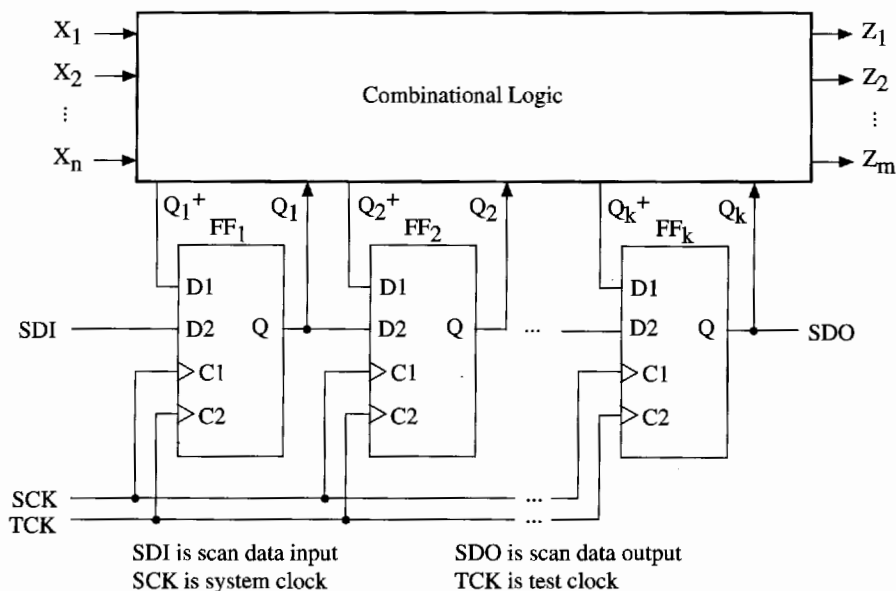
10.3 SCAN TESTING

The problem of testing a sequential network is greatly simplified if we can observe the state of all the flip-flops instead of just observing the network outputs. For each state of the flip-flops and for each input combination, we need to verify that the network outputs are correct and that the network goes to the correct next state. One approach would be to connect the output of each flip-flop within the IC being tested to one of the IC pins. Since the number of pins on the IC is very limited, this approach is not very practical. So the question arises, how can we observe the state of all the flip-flops without using up a large number of pins on the IC? If the flip-flops were arranged to form a shift register, then we could shift out the state of the flip-flops bit by bit using a single serial output pin on the IC. This leads to the concept of *scan path testing*.

Figure 10-8 shows a method of scan path testing based on two-port flip-flops. In the usual way, the sequential network is separated into a combinational logic part and a state register composed of flip-flops. Each of the flip-flops has two D inputs and two clock inputs. When $C1$ is pulsed, the $D1$ input is stored in the flip-flop. When $C2$ is pulsed, $D2$ is stored in the flip-flop. The Q output of each flip-flop is connected to the $D2$ input of the next flip-flop to form a shift register. The next state ($Q_1^+ Q_2^+ \dots Q_k^+$) generated by the combinational logic is loaded into the flip-flops when $C1$ is pulsed, and the new state ($Q_1 Q_2 \dots Q_k$) feeds back into the combinational logic. When the network is not being

tested, the system clock ($SCK = CI$) is used. A set of inputs ($X_1 X_2 \dots X_n$) is applied, the outputs ($Z_1 Z_2 \dots Z_m$) are generated, SCK is pulsed, and the network goes to the next state.

Figure 10-8 Scan Path Test Circuit Using Two-port Flip-flops



When the network is being tested, the flip-flops are set to a specified state by shifting the state code into the register using the scan data input (SDI) and the test clock (TCK). A test input vector ($X_1 X_2 \dots X_n$) is applied, the outputs ($Z_1 Z_2 \dots Z_m$) are verified, and SCK is pulsed to take the network to the next state. The next state is then verified by pulsing TCK to shift the state code out of the scan data register via the scan data output (SDO). This method reduces the problem of testing a sequential network to that of testing a combinational network. Any of the standard methods can be used to generate a set of test vectors for the combinational logic. Each test vector contains $(n + k)$ bits, since there are n X inputs and k state inputs to the combinational logic. The X part of the test vector is applied directly, and the Q part is shifted in via the SDI . In summary, the test procedure is as follows:

1. Scan in the test vector Q_i values via SDI using the test clock TCK .
2. Apply the corresponding test values to the X_i inputs.
3. After sufficient time for the signals to propagate through the combinational network, verify the output Z_i values.
4. Apply one clock pulse to the system clock SCK to store the new values of Q_i^+ into the corresponding flip-flops.
5. Scan out and verify the Q_i values by pulsing the test clock TCK .
6. Repeat steps 1 through 5 for each test vector.

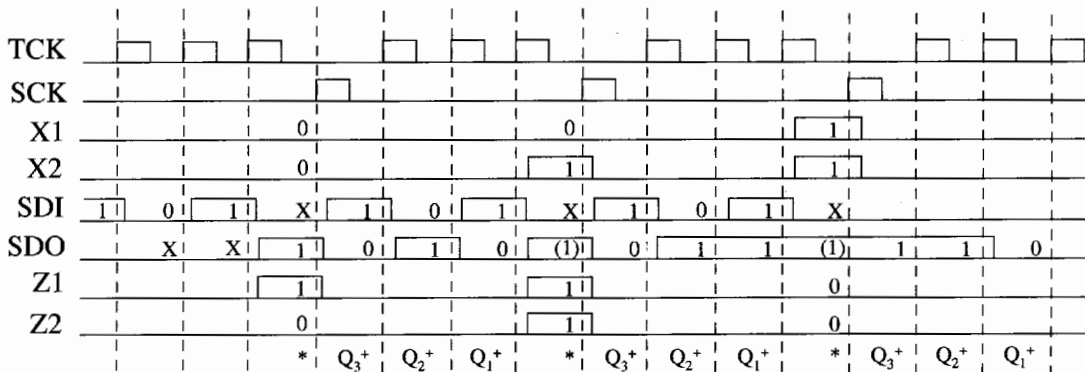
Steps 5 and 1 can overlap, since it is possible to scan in one test vector while scanning out the previous test result.

We will apply this method to test a sequential network with two inputs, three flip-flops, and two outputs. The network is configured as in Figure 10-8 with inputs X_1X_2 , flip-flops $Q_1Q_2Q_3$, and outputs Z_1Z_2 . One row of the state transition table is as follows:

$Q_1Q_2Q_3$	$Q_1^+Q_2^+Q_3^+$				Z_1Z_2			
	$X_1X_2 = 00 \ 01 \ 11 \ 10$				$00 \ 01 \ 11 \ 10$			
101	010 110 011 111				10 11 00 01			

Figure 10-9 shows the timing diagram for testing this row of the transition table. First, 101 is shifted in using *TCK*, least significant bit (Q_3) first. The input $X_1X_2 = 00$ is applied, and $Z_1Z_2 = 10$ is then read. *SCK* is pulsed and the network goes to state 010. As 010 is shifted out using *TCK*, 101 is shifted in for the next test. This process continues until the test is completed.

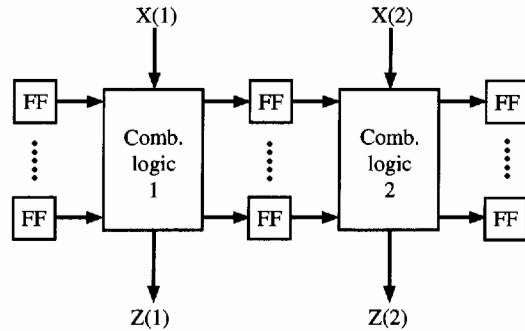
Figure 10-9 Timing Chart for Scan Test



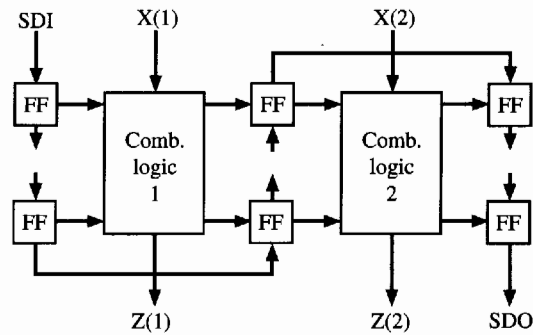
* Read output (output at other times not shown)

In general, a digital system implemented by an IC consists of flip-flop registers separated by blocks of combinational logic, as shown in Figure 10-10(a). In order to apply scan test to the IC, we need to replace the flip-flops with two-port flip-flops (or other types of scannable flip-flops) and link all the flip-flops into a scan chain, as shown in Figure 10-10(b). Then we can scan test data into all the registers, apply the test clock, and then scan out the results.

Figure 10-10 System with Flip-flop Registers and Combinational Logic Blocks



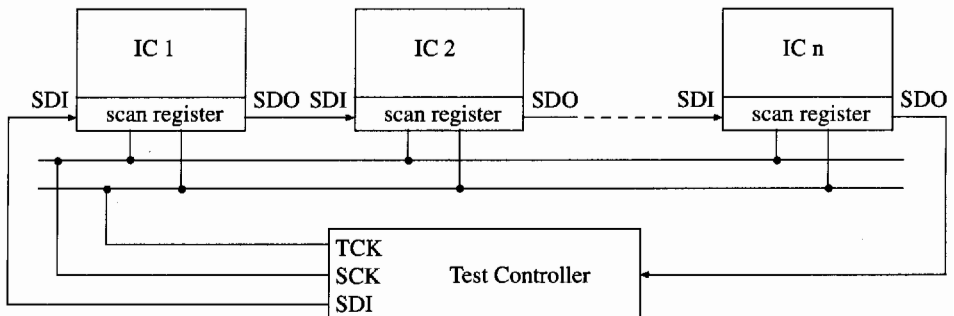
(a) Without scan chain



(b) With scan chain added

When multiple ICs are mounted on a PC board, it is possible to chain together the scan registers in each IC so that the entire board can be tested using a single serial access port (Figure 10-11).

Figure 10-11 Scan Test Configuration with Multiple ICs



10.4 BOUNDARY SCAN

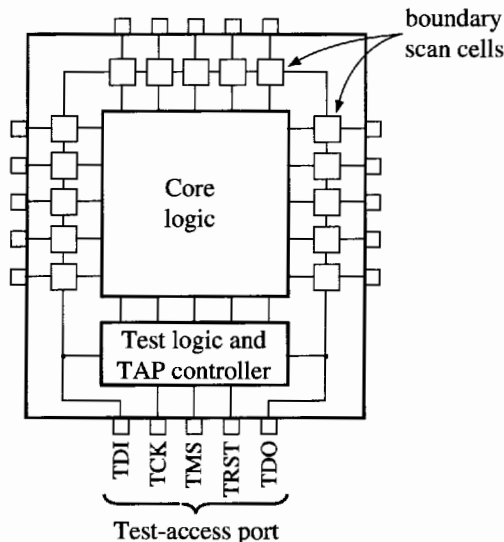
As ICs have become more complex, with more and more pins, printed circuit boards have become denser, with multiple layers and very fine traces. Testing these PC boards after they have been loaded with complex ICs has become very difficult. Testing a board by means of its edge connector does not provide adequate testing and may require very long test sequences. When PC boards were less dense with wider traces, testing was often done using a bed-of-nails test fixture. This method used sharp probes to contact the traces on the board so test data could be applied to and read from various ICs on the board. Bed-of-nails testing is not practical for high-density PC boards with fine traces and complex ICs.

Boundary scan test methodology was introduced to facilitate the testing of complex PC boards. A standard for boundary scan testing was developed by the Joint Test Action Group (JTAG), and this standard has been adopted as ANSI/IEEE Standard 1149.1, "Standard Test Access Port and Boundary-Scan Architecture." Many IC manufacturers make ICs that conform to this standard. Such ICs can be linked together on a PC board so that they can be tested using only a few pins on the PC board edge connector.

Figure 10-12 shows an IC with added boundary scan logic. One cell of the boundary scan register (BSR) is placed between each input or output pin and the internal core logic. Four or five pins of the IC are devoted to the test-access port, or TAP. The TAP controller and additional test logic are also added to the core logic on the IC. The functions of the TAP pins are as follows:

<i>TDI</i>	Test data input (this data is shifted serially into the BSR)
<i>TCK</i>	Test clock
<i>TMS</i>	Test mode select
<i>TDO</i>	Test data output (serial output from the BSR)
<i>TRST</i>	Test reset (resets the TAP controller and test logic; optional pin)

Figure 10-12 IC with Boundary Scan Register and Test-access Port



A PC board with several boundary scan ICs is shown in Figure 10-13. The boundary scan registers in the ICs are linked together serially in a single chain with input *TDI* and output *TDO*. *TCK*, *TMS*, and *TRST* (if used) are connected in parallel to all of the ICs. Using these signals, test instructions and test data can be clocked into every IC on the board.

Figure 10-13 PC Board with Boundary Scan ICs

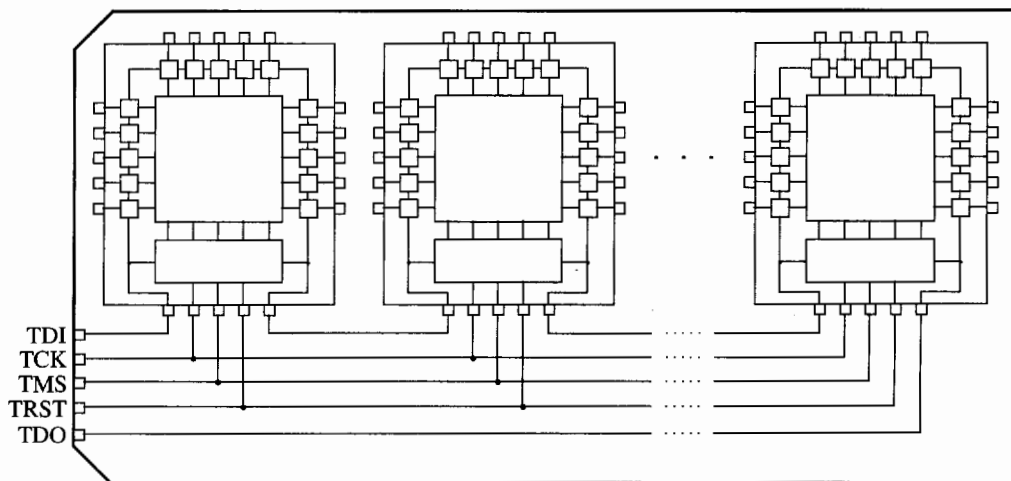


Figure 10-14 shows a typical boundary scan cell. When in the normal mode, data from the input pin is routed to the internal core logic in the IC, or data from the core logic is routed to the output pin. When in the shift mode, serial data from the previous cell is clocked into flip-flop *Q1* at the same time as the data stored in *Q1* is clocked into the next boundary scan cell. After *Q2* is updated, test data can be supplied to the internal logic or to the output pin.

Figure 10-14 Typical Boundary Scan Cell

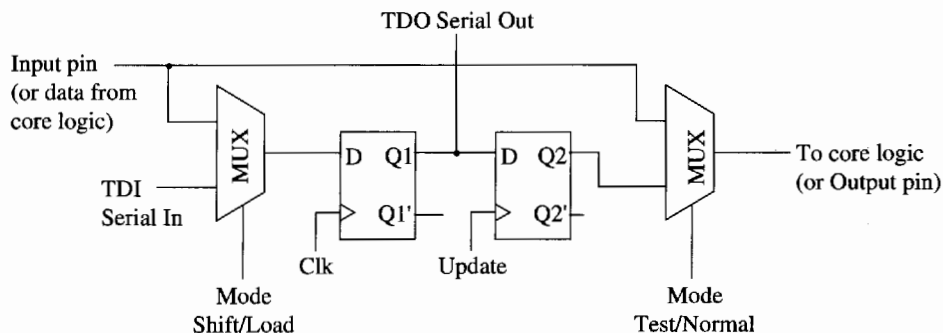


Figure 10-15 shows the basic boundary scan architecture that is implemented on each boundary scan IC. The boundary scan register is divided into two parts. BSR1 represents the shift register, which consists of the $Q1$ flip-flops in the boundary scan cells. BSR2 represents the $Q2$ flip-flops, which can be parallel-loaded from BSR1 when an update signal is received. The serial input data (TDI) can be shifted into the boundary scan register (BSR1), through a bypass register, or into the instruction register. The TAP controller on each IC contains a state machine (Figure 10-16). The input to the state machine is TMS , and the sequence of 0s and 1s applied to TMS determines whether the TDI data is shifted into the instruction register or through the boundary scan cells. The TAP controller and the instruction register control the operation of the boundary scan cells.

Figure 10-15 Basic Boundary Scan Architecture

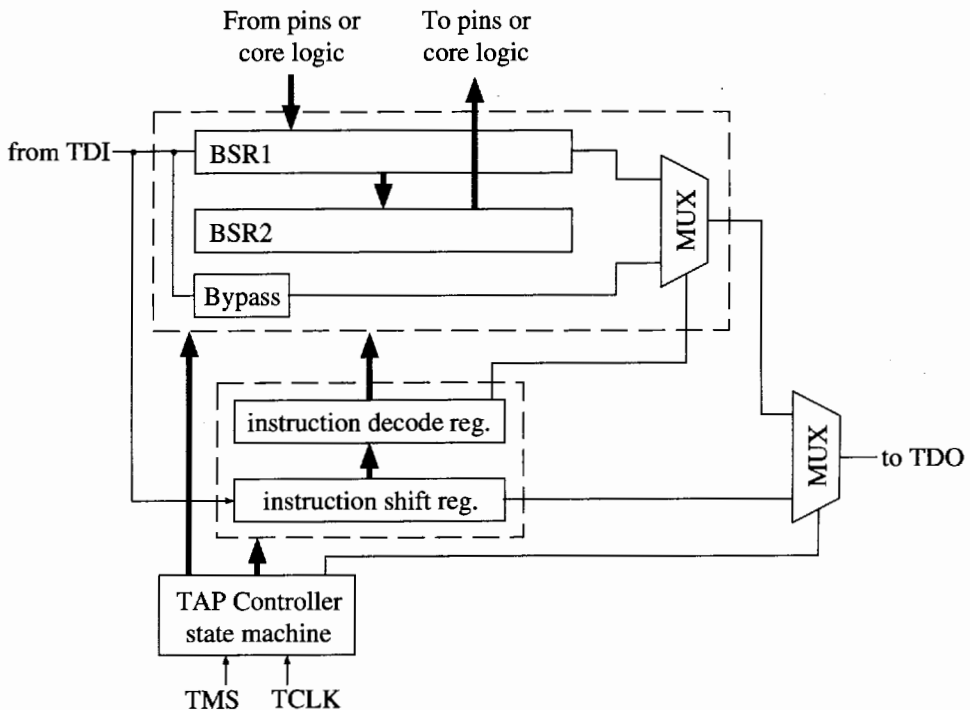
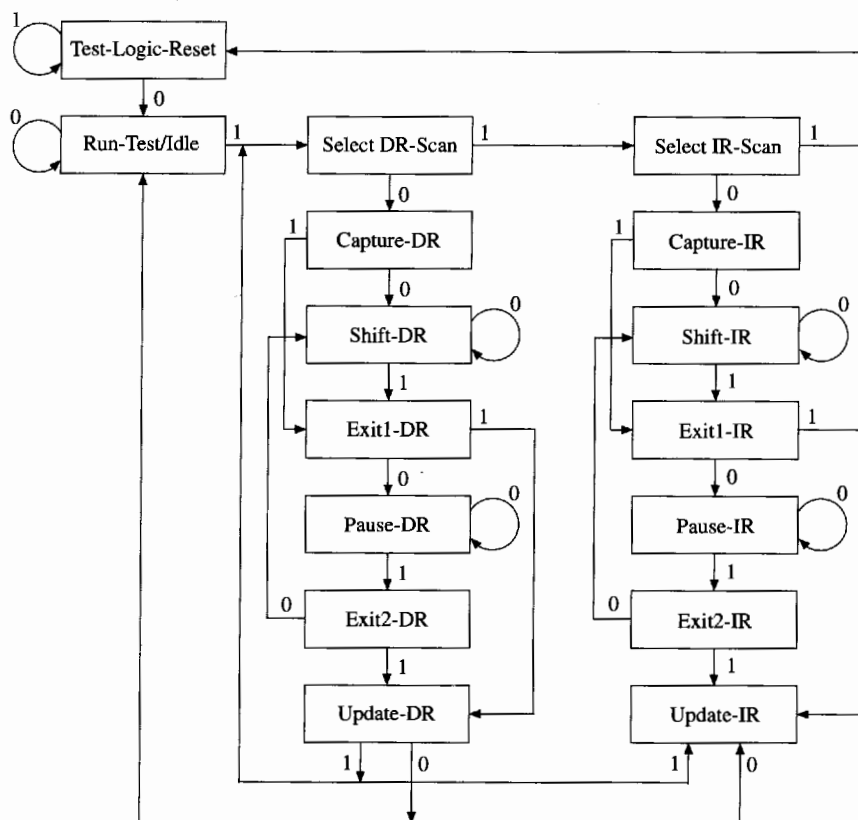


Figure 10-16 State Machine for TAP Controller



The following instructions are defined in the IEEE standard:

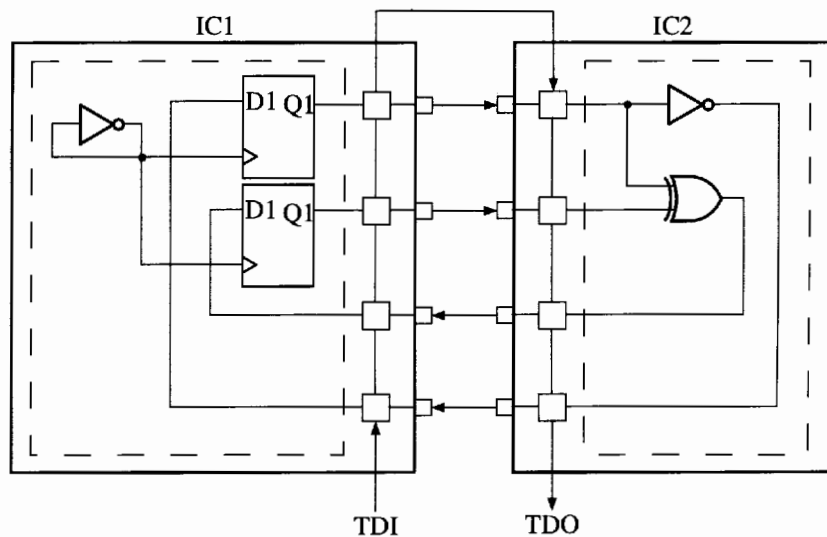
- **BYPASS:** This instruction allows the *TDI* serial data to go through a 1-bit bypass register on the IC instead of through the boundary scan register. In this way, one or more ICs on the PC board may be bypassed while other ICs are being tested.
- **SAMPLE/PRELOAD:** This instruction is used to scan the boundary scan register without interfering with the normal operation of the core logic. Data is transferred to or from the core logic from or to the IC pins without interference. Samples of this data can be taken and scanned out through the boundary scan register. Test data can be shifted into the BSR.
- **EXTEST:** This instruction allows board-level interconnect testing, and it also allows testing of clusters of components that do not incorporate the boundary scan test features. Test data is shifted into the BSR and then it goes to the output pins. Data from the input pins is captured by the BSR.
- **INTEST (optional):** This instruction allows testing of the core logic by shifting test data into the boundary scan register. Data shifted into the BSR takes the place of data from the input pins, and output data from the core logic is loaded into the BSR.

- **RUNBIST** (optional): This instruction causes special built-in self-test (BIST) logic within the IC to execute. (Section 10.5 explains how BIST logic can be used to generate test sequences and check the test results.)

Several other optional and user-defined instructions may also be included.

The following simplified example illustrates how the connections between two ICs can be tested using the **SAMPLE/PRELOAD** and **EXTEST** instructions. The test is intended to check for shorts and opens in the PC board traces. Both ICs have two input pins and two output pins, as shown in Figure 10-17. Test data is shifted into the BSRs via *TDI*. Then data from the input pins is parallel-loaded into the BSRs and shifted out via *TDO*. We assume that the instruction register on each IC is three bits long with **EXTEST** coded as 000 and **SAMPLE/PRELOAD** as 001. The core logic in IC1 is an inverter connected as a clock oscillator and two flip-flops. The core logic in IC2 is an inverter and XOR gate. The two ICs are interconnected to form a 2-bit counter.

Figure 10-17 Interconnection Testing Using Boundary Scan



The steps required to test the connections between the ICs are as follows:

1. Reset the TAP state machine to the Test-Logic-Reset state by inputting a sequence of five 1s on *TMS*. The TAP controller is designed so that a sequence of five 1s will always reset it, regardless of the present state. Alternatively, *TRST* can be asserted if it is available.
2. Scan in the **SAMPLE/PRELOAD** instruction to both ICs using the sequences for *TMS* and *TDI* given here. The state numbers refer to Figure 10-16.

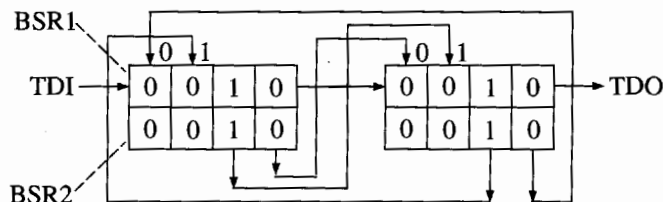
State:	0	1	2	9	10	11	11	11	11	11	11	12	15	2
TMS:	0	1	1	0	0	0	0	0	0	0	1	1	1	
TDI:	-	-	-	-	-	1	0	0	1	0	0	-	-	

The *TMS* sequence 01100 takes the TAP controller to the Shift-IR state. In this state, copies of the SAMPLE/PRELOAD instruction (code 001) are shifted into the instruction registers on both ICs. In the Update-IR state, the instructions are loaded into the instruction decode registers. Then the TAP controller goes back to the Select DR-scan state.

3. Preload the first set of test data into the ICs using the following sequences for *TMS* and *TDI*:

```
State: 2 3 4 4 4 4 4 4 4 4 5 8 2
TMS:   0 0 0 0 0 0 0 0 0 1 1 1
TDI:   - - 0 1 0 0 0 1 0 0 - -
```

Data is shifted into BSR1 in the Shift-DR state, and it is transferred to BSR2 in the Update-DR state. The result is as follows:



4. Scan in the EXTEST instruction to both ICs using the following sequences:

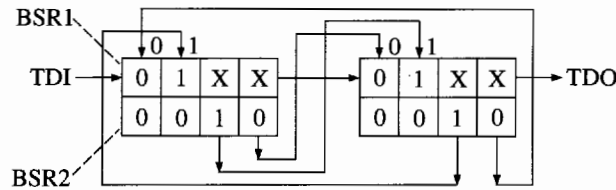
```
State: 2 9 10 11 11 11 11 11 11 12 15 2
TMS:   1 0 0 0 0 0 0 0 0 1 1 1
TDI:   - - - 0 0 0 0 0 0 0 - -
```

The EXTEST instruction (000) is scanned into the instruction register in state Shift-IR and loaded into the instruction decode register in state Update-IR. At this point, the preloaded test data goes to the output pins, and it is transmitted to the adjacent IC input pins via the printed circuit board traces.

5. Capture the test results from the IC inputs. Scan this data out to *TDO* and scan the second set of test data in using the following sequences:

```
State: 2 3 4 4 4 4 4 4 4 4 5 8 2
TMS:   0 0 0 0 0 0 0 0 0 1 1 1
TDI:   - - 1 0 0 0 1 0 0 0 - -
TDO:   - - x x 1 0 x x 1 0 - -
```

The data from the input pins is loaded into BSR1 in state Capture-DR. At this time, if no faults have been detected, the BSRs should be configured as shown below, where the Xs indicate captured data that is not relevant to the test.



The test results are then shifted out of BSR1 in state Shift-DR as the new test data is shifted in. The new data is loaded into BSR2 in the Update-IR state.

6. Capture the test results from the IC inputs. Scan this data out to *TDO* and scan all 0s in using the following sequences:

```

State: 2 3 4 4 4 4 4 4 4 4 5 8 2 9 0
TMS:   0 0 0 0 0 0 0 0 0 0 1 1 1 1 1
TDI:   - - 0 0 0 0 0 0 0 0 - - - -
TDO:   - - x x 0 1 x x 0 1 - - - -

```

The data from the input pins is loaded into BSR1 in state Capture-DR. Then it is shifted out in state Shift-DR as all 0s are shifted in. The 0s are loaded into BSR2 in the Update-IR state. The controller then returns to the Test-Logic-Reset state, and normal operation of the ICs can then occur. The interconnection test passes if the observed *TDO* sequences match the ones given above.

VHDL code for the basic boundary scan architecture of Figure 10-15 is given in Figure 10-18. Only the three mandatory instructions (EXTEST, SAMPLE/PRELOAD, and BYPASS) are implemented using a 3-bit instruction register. These instructions are coded as 000, 001, and 111, respectively. The number of cells in the *BSR* is a generic parameter. A second generic parameter, *CellType*, is a bit_vector that specifies whether each cell is an input cell or output cell. The case statement implements the TAP controller state machine. The instruction code is scanned in and loaded into *IDR* in states Capture-IR, Shift-IR, and Update-IR. The instructions are executed in states Capture-DR, Shift-DR, and Update-DR. The actions taken in these states depend on the instruction being executed. The register updates and state changes all occur on the rising edge of *TCK*. The VHDL code implements most of the functions required by the IEEE boundary scan standard, but it does not fully comply with the standard.

Figure 10-18 VHDL Code for Basic Boundary Scan Architecture

```

-- VHDL for Boundary Scan Architecture of Figure 10-15

entity BS_arch is
  generic (NCELLS: natural range 2 to 120 := 2);
  -- number of boundary scan cells
  port (TCK, TMS, TDI: in bit;
        TDO: out bit;
        BSRin: in bit_vector(1 to NCELLS);
        BSRout: inout bit_vector(1 to NCELLS);
        CellType: bit_vector(1 to NCELLS));
  -- '0' for input cell, '1' for output cell
end BS_arch;

```

```

architecture behavior of BS_arch is
    signal IR, IDR: bit_vector(1 to 3);           -- instruction registers
    signal BSR1, BSR2: bit_vector(1 to NCELLS);   -- boundary scan cells
    signal BYPASS: bit;                           -- bypass bit
    type TAPstate is (TestLogicReset, RunTest_Idle,
        SelectDRScan, CaptureDR, ShiftDR, Exit1DR, PauseDR, Exit2DR, UpdateDR,
        SelectIRScan, CaptureIR, ShiftIR, Exit1IR, PauseIR, Exit2IR, UpdateIR);
    signal St: TAPstate;                          -- TAP Controller State
begin
    process (TCK)
    begin
        if (TCK='1') then
            -- TAP Controller State Machine
            case St is
                when TestLogicReset =>
                    if TMS='0' then St<=RunTest_Idle; else St<=TestLogicReset; end if;
                when RunTest_Idle =>
                    if TMS='0' then St<=RunTest_Idle; else St<=SelectDRScan; end if;
                when SelectDRScan =>
                    if TMS='0' then St<=CaptureDR; else St<=SelectIRScan; end if;
                when CaptureDR =>
                    if IDR = "111" then BYPASS <= '0';
                    elsif IDR = "000" then -- EXTEST (input cells capture pin data)
                        BSR1 <= (not CellType and BSRin) or (CellType and BSR1);
                    elsif IDR = "001" then -- SAMPLE/PRELOAD
                        BSR1 <= BSRin; end if; -- all cells capture cell input data
                    if TMS='0' then St<=ShiftDR; else St<=Exit1DR; end if;
                when ShiftDR =>
                    if IDR = "111" then BYPASS <= TDI; -- shift data through bypass reg.
                    else BSR1 <= TDI & BSR1(1 to NCELLS-1); end if;
                    -- shift data into BSR
                    if TMS='0' then St<=ShiftDR; else St<=Exit1DR; end if;
                when Exit1DR =>
                    if TMS='0' then St<=PauseDR; else St<=UpdateDR; end if;
                when PauseDR =>
                    if TMS='0' then St<=PauseDR; else St<=Exit2DR; end if;
                when Exit2DR =>
                    if TMS='0' then St<=ShiftDR; else St<=UpdateDR; end if;
                when UpdateDR =>
                    if IDR = "000" then -- EXTEST (update output reg. for output cells)
                        BSR2 <= (CellType and BSR1) or (not CellType and BSR2);
                    elsif IDR = "001" then -- SAMPLE/PRELOAD
                        BSR2 <= BSR1; end if; -- update output reg. in all cells
                    if TMS='0' then St<=RunTest_Idle; else St<=SelectDRScan; end if;
                when SelectIRScan =>
                    if TMS='0' then St<=CaptureIR; else St<=TestLogicReset; end if;
                when CaptureIR =>
                    IR <= "001"; -- load 2 LSBs of IR with 01 as required by the
                        standard
                    if TMS='0' then St<=ShiftIR; else St<=Exit1IR; end if;
            end case;
        end if;
    end process;
end

```

```

when ShiftIR =>
    IR <= TDI & IR(1 to 2); -- shift in instruction code
    if TMS='0' then St<=ShiftIR;      else St<=Exit1IR;      end if;
when Exit1IR =>
    if TMS='0' then St<=PauseIR;      else St<=UpdateIR;      end if;
when PauseIR =>
    if TMS='0' then St<=PauseIR;      else St<=Exit2IR;      end if;
when Exit2IR =>
    if TMS='0' then St<=ShiftIR;      else St<=UpdateIR;      end if;
when UpdateIR =>
    IDR <= IR;    -- update instruction decode register
    if TMS='0' then St<=RunTest_Idle; else St<=SelectDRScan; end if;
end case;
end if;
end process;

TDO <= BYPASS when St = ShiftDR and IDR = "111" -- BYPASS
      else BSR1(NCELLS) when St=ShiftDR -- EXTEST or SAMPLE/PRELOAD
      else IR(3) when St=ShiftIR;

BSRout <= BSRin when (St = TestLogicReset or not (IDR = "000"))
      else BSR2; -- define cell outputs
end behavior;

```

VHDL code that implements the interconnection test example of Figure 10-17 is given in Figure 10-19. The *TMS* and *TDI* test patterns are the concatenation of the test patterns used in steps 2 through 6. A copy of the basic boundary scan architecture is instantiated for IC1 and for IC2. The external connections and internal logic for each IC are then specified. The internal clock frequency was arbitrarily chosen to be different than the test clock frequency. The test process runs the internal logic, then runs the scan test, and then runs the internal logic again. The test results verify that the IC logic runs correctly and that the scan test produces the expected results.

Figure 10-19 VHDL Code for Interconnection Test Example

```

-- Boundary Scan Tester

entity system is
end system;

architecture IC_test of system is
    component BS_arch is
        generic (NCELLS:natural range 2 to 120 := 4);
        port (TCK, TMS, TDI: in bit;
              TDO: out bit;
              BSRin: in bit_vector(1 to NCELLS);
              BSRout: inout bit_vector(1 to NCELLS);
              CellType in bit_vector(1 to NCELLS));
        -- '0' for input cell, '1' for output cell
    end component;

```

```

signal TCK,TMS,TDI,TDO,TDO1: bit;
signal Q0, Q1, CLK1: bit;
signal BSR1in, BSR1out, BSR2in, BSR2out: bit_vector(1 to 4);
signal count: integer := 0;

constant TMSpattern: bit_vector(0 to 62) :=
    "011000000011100000000011110000000111000000000111000000000111111";
constant TDIPattern: bit_vector(0 to 62) :=
    "0000010010000000100010000000000000000000100010000000000000000000";
begin
    BS1: BS_arch port map(TCK, TMS, TDI, TDO1, BSR1in, BSR1out, "0011");
    BS2: BS_arch port map(TCK, TMS, TDO1, TDO, BSR2in, BSR2out, "0011");
    -- each BSR has two input cells and two output cells

    BSR1in(1) <= BSR2out(4);           -- IC1 external connections
    BSR1in(2) <= BSR2out(3);
    BSR1in(3) <= Q1;                   -- IC1 internal logic
    BSR1in(4) <= Q0;
    CLK1 <= not CLK1 after 7 ns;       -- internal clock
    process(CLK1)
    begin
        if (CLK1='1') then             -- D flip-flops
            Q0 <= BSR1out(1);
            Q1 <= BSR1out(2);
        end if;
    end process;

    BSR2in(1) <= BSR1out(4);           -- IC2 external connections
    BSR2in(2) <= BSR1out(3);
    BSR2in(3) <= BSR2out(1) xor BSR2out(2); -- IC2 internal logic
    BSR2in(4) <= not BSR2out(1);

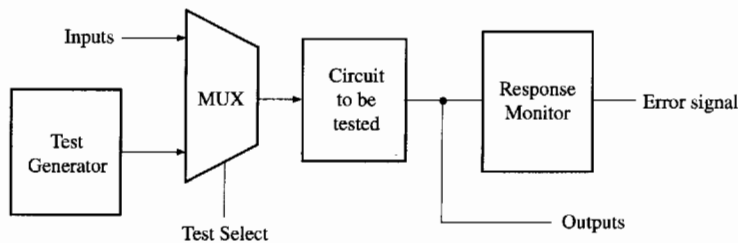
    TCK <= not TCK after 5 ns;         -- test clock
    process
    begin
        TMS <= '1';
        wait for 70 ns;                 -- run internal logic
        wait until TCK='1';
        for i in TMSpattern'range loop -- run scan test
            TMS <= TMSpattern(i);
            TDI <= TDIPattern(i);
            wait for 0 ns;
            count <= i+1;               -- count triggers listing output
            wait until TCK='1';
        end loop;
        wait for 70 ns;                 -- run internal logic
        wait;                           -- stop
    end process;
end IC_test;

```

10.5 BUILT-IN SELF-TEST

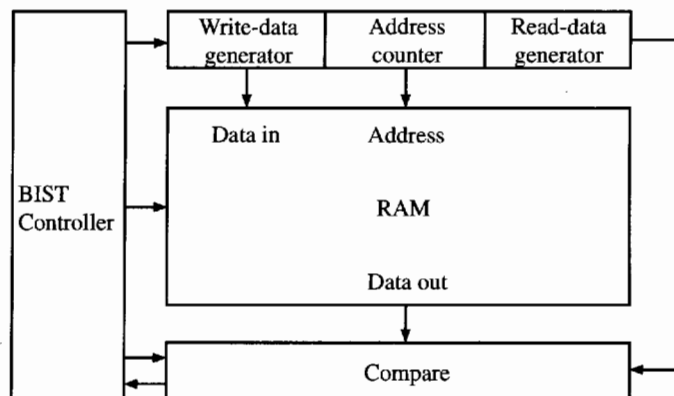
As digital systems become more and more complex, they become much harder and more expensive to test. One solution to this problem is to add logic to the IC so that it can test itself. This is referred to as Built-In Self-Test, or BIST. Figure 10-20 illustrates the general method for using BIST. When the test mode is selected by the test-select signal, an on-chip test generator applies test patterns to the circuit under test. The resulting output is observed by the response monitor, which produces an error signal if an incorrect output pattern is detected.

Figure 10-20 Generic BIST Scheme



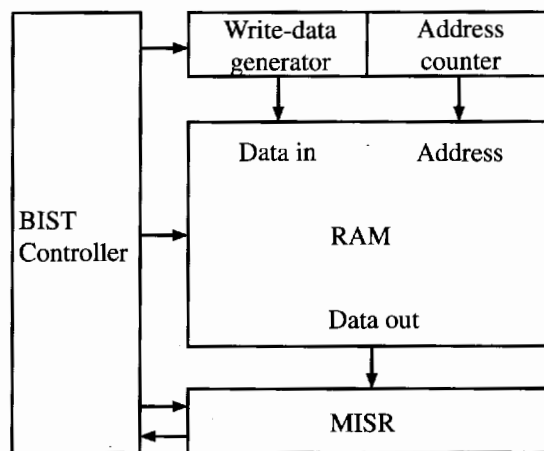
BIST is often used for testing memory. The regular structure of a memory chip makes it easy to generate test patterns. Figure 10-21 shows a block diagram of a self-test circuit for a RAM. The BIST controller enables the write-data generator and address counter so that data is written to each location in the RAM. Then the address counter and read-data generator are enabled, and the data read from each RAM location is compared with the output of the read-data generator to verify that it is correct.

Figure 10-21 Self-test Circuit for RAM



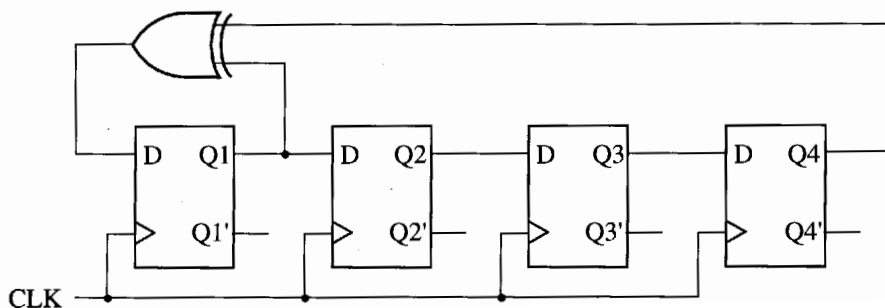
The test circuit can be simplified by using a signature register. The signature register compresses the output data into a short string of bits called a *signature*, and this signature is compared with the signature for a correctly functioning component. A multiple-input signature register (MISR) combines and compresses several output streams into a single signature. Figure 10-22 shows a simplified version of the RAM self-test circuit. The read-data generator and comparator have been eliminated and replaced with a MISR. One type of MISR simply forms a check sum by adding up all the data bytes stored in the RAM. When testing a ROM, Figure 10-22 can be simplified further, since no write-data generator is needed.

Figure 10-22 Self-test Circuit for RAM with Signature Register



Linear feedback shift registers (LFSRs) are often used to generate test patterns. Figure 10-23 shows an example of a LFSR. The outputs from the first and fourth flip-flops are XORed together and fed back into the *D* input of the first flip-flop. The general form of a LFSR is a shift register with two or more flip-flop outputs XORed together and fed back into the first flip-flop. The name *linear* comes from the fact that exclusive OR is equivalent to modulo-2 addition, and addition is a linear operation.

Figure 10-23 4-bit Linear Feedback Shift Register (LFSR)



By proper choice of the outputs that are fed back through the exclusive OR gate, it is possible to generate $2^n - 1$ different bit patterns using an n -bit shift register. All possible patterns can be generated except for all 0s. The patterns generated by the LFSR of Figure 10-23 are:

1000, 1100, 1110, 1111, 0111, 1011, 0101, 1010, 1101, 0110, 0011,
1001, 0100, 0010, 0001, 1000, . . .

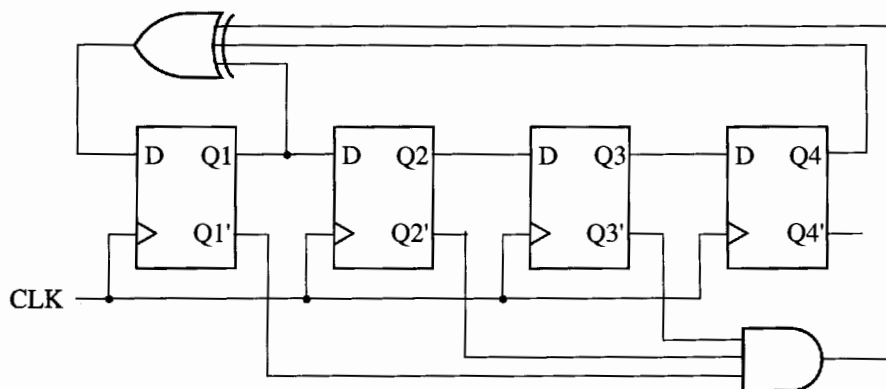
These patterns have no obvious order, and they have certain randomness properties. Such a LFSR is often referred to as a *pseudo-random pattern generator*, or PRPG. PRPGs are obviously very useful for BIST, since they can generate a large number of test patterns with a small amount of logic circuitry. Table 10-4 gives one feedback combination that will generate all $2^n - 1$ bit patterns for some LFSRs with lengths in the range $n = 4$ to 32.

Table 10-4 Feedback for Maximum-length LFSR Sequence

n	Feedback
4,6,7	$Q_1 \oplus Q_n$
5	$Q_1 \oplus Q_2 \oplus Q_5$
8	$Q_2 \oplus Q_3 \oplus Q_4 \oplus Q_8$
12	$Q_1 \oplus Q_4 \oplus Q_6 \oplus Q_{12}$
14,16	$Q_3 \oplus Q_4 \oplus Q_5 \oplus Q_n$
24	$Q_1 \oplus Q_2 \oplus Q_7 \oplus Q_{24}$
32	$Q_1 \oplus Q_2 \oplus Q_{22} \oplus Q_{32}$

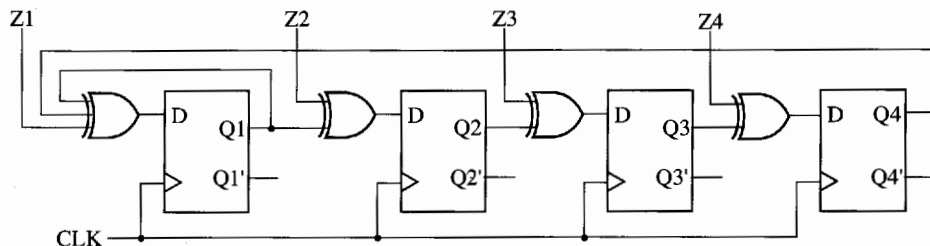
If the all-0s test pattern is required, an n -bit LFSR can be modified by adding an AND gate with $n - 1$ inputs, as shown in Figure 10-24 for $n = 4$. When in state 0001, the next state is 0000; when in state 0000, the next state is 1000; otherwise, the sequence is the same.

Figure 10-24 Modified LFSR with 0000 State



A MISR can be constructed by modifying a LFSR by adding XOR gates, as shown in Figure 10-25. The test data ($Z_1Z_2Z_3Z_4$) is XORed into the register with each clock, and the final result represents a signature that can be compared with the signature for a known correctly functioning component. This type of signature analysis will catch many, but not all, possible errors. An n -bit signature register maps all possible input streams into one of the 2^n possible signatures. One of these is the correct signature, and the others indicate that errors have occurred. The probability that an incorrect input sequence will map to the correct signature is of the order of $1/2^n$.

Figure 10-25 Multiple-input Signature Register (MISR)



For the MISR of Figure 10-25, assume that the correct input sequence is 1010, 0001, 1110, 1111, 0100, 1011, 1001, 1000, 0101, 0110, 0011, 1101, 0111, 0010, 1100. This sequence maps to the signature 1010. Any sequence that differs in one bit will map to a different signature. For example, if 0001 in the sequence is changed to 1001, the resulting sequence maps to 1000. Most sequences with two errors will be detected, but if we change 0001 to 1001 and 0010 to 0110 in the original sequence, the result maps to 1010, which is the correct signature, so the errors would not be detected.

To adapt the scan test scheme of Figure 10-10(b) for BIST, the scan register is modified so each part of the register can serve as a state register, pattern generator, signature register, or shift register. When used as a shift register, the test data can be scanned in and out in the usual way. Then part of the scan register can be used as a pattern generator (PRPG) and part as a signature register (MISR) to test one of the combinational blocks. The roles can then be changed to test another combinational block. When the testing is finished, the scan register is placed in the state register mode for normal operation.

One such scheme is referred to as BILBO (built-in logic block observation). Figure 10-26 shows the placement of BILBO registers for testing a network with two combinational blocks. Comb. network 1 is tested when the first BILBO is used as a PRPG and the second as a MISR. The roles of the registers are reversed to test Comb. network 2. In the normal operating mode, both BILBOs serve as registers for the associated combinational logic. To scan data in and out, both BILBOs operate in the shift register mode.

Figure 10-26 BIST Using BILBO Registers

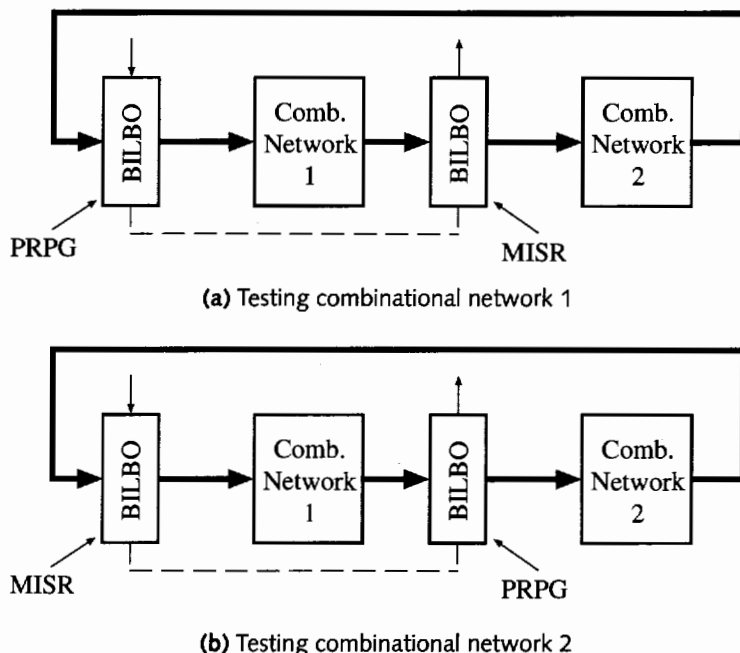


Figure 10-27 shows the structure of one version of a 4-bit BILBO register. The control inputs B_1 and B_2 determine the operating mode. Si and So are the serial input and output for the shift register mode. The Z s are inputs from the combinational logic. The equations for this BILBO register are

$$D_1 = Z_1 B_1 \oplus (Si B_2' + FB B_2) (B_1' + B_2)$$

$$D_i = Z_i B_1 \oplus Q_{i-1} (B_1' + B_2) \quad (i > 1)$$

When $B_1 = B_2 = 0$, these equations reduce to

$$D_1 = Si \text{ and } D_i = Q_{i-1} \quad (i > 1)$$

which corresponds to the shift register mode. When $B_1 = 0$ and $B_2 = 1$, the equations reduce to

$$D_1 = FB, \quad D_i = Q_{i-1}$$

which corresponds to the PRPG mode, and the BILBO register is equivalent to Figure 10-23. When $B_1 = 1$ and $B_2 = 0$, the equations reduce to

$$D_1 = Z_1, \quad D_i = Z_i$$

which corresponds to the normal operating mode. When $B_1 = B_2 = 1$, the equations reduce to

$$D_1 = Z_1 \oplus FB, \quad D_i = Z_i \oplus Q_{i-1}$$

which corresponds to the MISR mode, and the BILBO register is equivalent to Figure 10-25. In summary, the BILBO operating modes are as follows:

B1B2	Operating Mode
00	shift register
01	PRPG
10	normal
11	MISR

Figure 10-27 4-bit BILBO Register

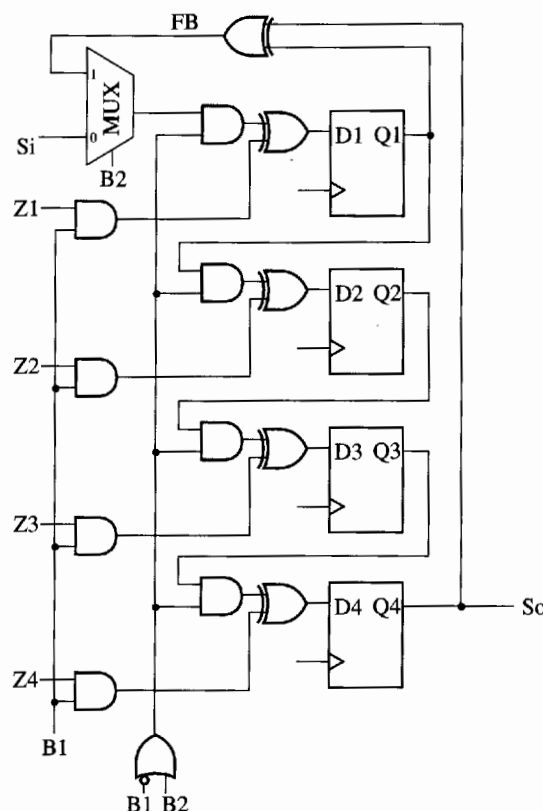


Figure 10-28 shows the VHDL description of an n -bit BILBO register. NBITS, which equals the number of bits, is a generic parameter in the range 4 through 8. The register is functionally equivalent to Figure 10-27, except that we have added a clock enable (CE). The feedback (FB) for the LFSR depends on the number of bits.

The system shown in Figure 10-29 illustrates the use of BILBO registers. In this system, registers A and B can be loaded from the Dbus using the LDA and LDB signals. Then the registers are added and the sum and carry are stored in register C . When $B1B2 = 10$, the registers are in the normal mode ($Test = 0$), and loading of the registers is controlled by

LDA, *LDB*, and *LDC*. To test the adder, we first set *BIB2* = 00 to place the registers in the shift register mode and scan in initial values for *A*, *B*, and *C*. Then we set *BIB2* = 10, which places registers *A* and *B* in PRPG mode and register *C* in MISR mode. After 15 clocks, the test is complete. Then we can set *BIB2* = 00 and scan out the signature.

Figure 10-28 VHDL Code for BILBO Register of Figure 10-27

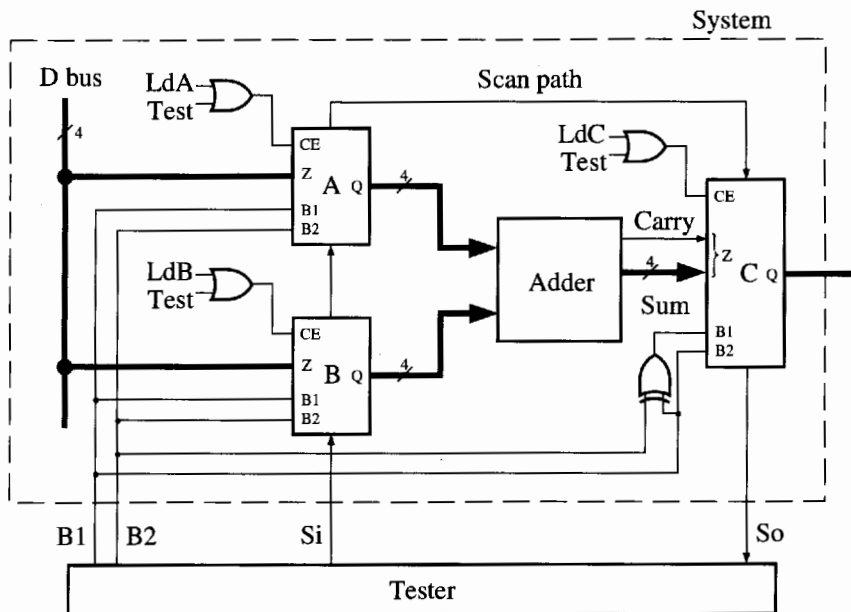
```

entity BILBO is                                -- BILBO Register
  generic (NBITS: natural range 4 to 8 := 4);
  port (Clk, CE, B1, B2, Si: in bit;
        So: out bit;
        Z: in bit_vector(1 to NBITS);
        Q: inout bit_vector(1 to NBITS));
end BILBO;

architecture behavior of BILBO is
  signal FB: bit;
begin
  FB <= Q(2) xor Q(3) xor Q(4) xor Q(NBITS) when (NBITS=8)
    else Q(2) xor Q(NBITS) when (NBITS=5)
    else Q(1) xor Q(NBITS);
  process(Clk)
    variable mode: bit_vector(1 downto 0);
  begin
    if (Clk = '1' and CE = '1') then
      mode := B1 & B2;
      case mode is
        when "00" =>                                -- Shift register mode
          Q <= Si & Q(1 to NBITS-1);
        when "01" =>                                -- Pseudo Random Pattern Generator mode
          Q <= FB & Q(1 to NBITS-1);
        when "10" =>                                -- Normal Operating mode
          Q <= Z;
        when "11" =>                                -- Multiple Input Signature Register mode
          Q <= Z(1 to NBITS) xor (FB & Q(1 to NBITS-1));
      end case;
    end if;
  end process;
  So <= Q(NBITS);
end;

```

Figure 10-29 System with BILBO Registers and Tester



The VHDL code for the system is given in Figure 10-30, and a test bench is given in Figure 10-31. The system uses three BILBO registers and the 4-bit adder of Figure 8-12. The test bench scans in a test vector to initialize the BILBO registers, then it runs the test with registers *A* and *B* used as PRPGs and register *C* as a MISR. The resulting signature is shifted out and compared with the correct signature.

Figure 10-30 VHDL Code for System with BILBO Registers and Tester

```
entity BILBO_System is
  port (Clk, LdA, LdB, LdC, B1, B2, Si: in bit;
        So: out bit;
        DBus: in bit_vector(3 downto 0);
        Output: inout bit_vector(4 downto 0));
end BILBO_System;
```

```

architecture BSys1 of BILBO_System is
  component Adder4 is
    port (A, B: in bit_vector(3 downto 0); Ci: in bit;
          S: out bit_vector(3 downto 0); Co: out bit);
  end component;
  component BILBO is
    generic (NBITS: natural range 4 to 8 := 4);
    port (Clk, CE, B1, B2, Si : in bit;
          So: out bit;
          Z: in bit_vector(1 to NBITS);
          Q: inout bit_vector(1 to NBITS));
  end component;

  signal Aout, Bout: bit_vector(3 downto 0);
  signal Cin: bit_vector(4 downto 0);
  alias Carry: bit is Cin(4);
  alias Sum: bit_vector is Cin(3 downto 0);
  signal ACE, BCE, CCE, CB1, Test, S1, S2: bit;
begin
  Test <= not B1 or B2;
  ACE <= Test or LdA;
  BCE <= Test or LdB;
  CCE <= Test or LdC;
  CB1 <= B1 xor B2;
  RegA: BILBO generic map (4) port map(Clk, ACE, B1, B2, S1, S2, DBus,
    Aout);
  RegB: BILBO generic map (4) port map(Clk, BCE, B1, B2, Si, S1, DBus,
    Bout);
  RegC: BILBO generic map (5) port map(Clk, CCE, CB1, B2, S2, So, Cin,
    Output);
  Adder: Adder4 port map(Aout, Bout, '0', Sum, Carry);
end BSys1;

```

Figure 10-31 Test Bench for BILBO System

```

-- System with BILBO test bench

entity BILBO_test is
end BILBO_test;

architecture Btest of BILBO_test is
  component BILBO_System is
    port (Clk, LdA, LdB, LdC, B1, B2, Si: in bit;
          So: out bit;
          DBus: in bit_vector(3 downto 0);
          Output: inout bit_vector(4 downto 0));
  end component;

```

```

signal Clk: bit := '0';
signal LdA, LdB, LdC, B1, B2, Si, So: bit := '0';
signal DBus: bit_vector(3 downto 0);
signal Output: bit_vector(4 downto 0);
signal Sig: bit_vector(4 downto 0);

constant test_vector: bit_vector(12 downto 0) := "10001100000000";
constant test_result: bit_vector(4 downto 0) := "01011";
begin
  clk <= not clk after 10 ns;
  Sys: BILBO_System port map (Clk, Lda, LdB, LdC, B1, B2, Si, So, DBus, Output);
  process
  begin
    B1 <= '0'; B2 <= '0';           -- Shift in test vector
    for i in test_vector'right to test_vector'left loop
      Si <= test_vector(i);
      wait until (clk = '1');
    end loop;

    B1 <= '0'; B2 <= '1';           -- Use PRPG and MISR
    for i in 1 to 15 loop
      wait until (clk = '1');
    end loop;

    B1 <= '0'; B2 <= '0';           -- Shift signature out
    for i in 0 to 5 loop
      Sig <= So & Sig(4 downto 1);
      wait until (clk = '1');
    end loop;

    if (Sig = test_result) then     -- Compare signature
      report "System passed test.";
    else
      report "System did not pass test!";
    end if;

    wait;
  end process;
end Btest;

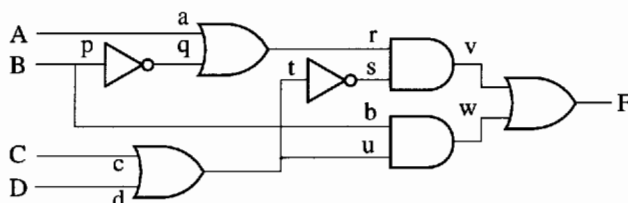
```

In this chapter, we introduced the subject of testing hardware, including combinational networks, sequential networks, complex ICs, and PC boards. Use of scan techniques for testing and built-in self-test has become a necessity as digital systems have become more complex. It is very important that design for testability be considered early in the design process so that the final hardware can be tested efficiently and economically.

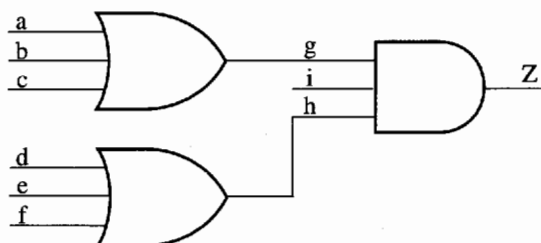
Problems

10.1

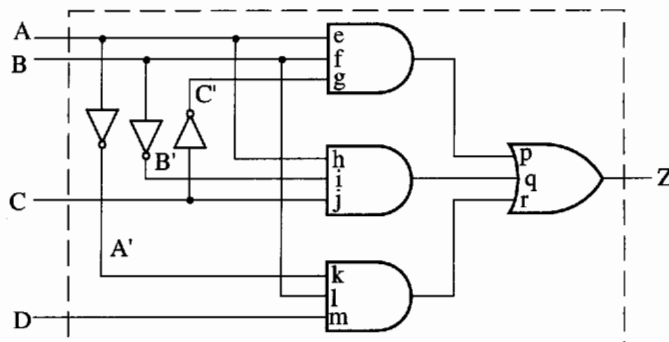
- Determine the necessary inputs to the following network to test for u stuck-at-0.
- For this set of inputs, determine which other stuck-at faults can be tested.
- Repeat (a) and (b) for r stuck-at-1.



- 10.2** Find a minimum set of tests that will test all single stuck-at-0 and stuck-at-1 faults in the following network. For each test, specify which faults are tested for s-a-0 and for s-a-1.



- 10.3** For the following network, find a minimum number of test vectors that will test all s-a-0 and s-a-1 faults at the AND and OR gate inputs. For each test vector, specify the values of A , B , C and D , and the stuck-at faults that are tested.



- 10.4** Find a test sequence to test for b s-a-0 in the sequential network of Figure 10-7.

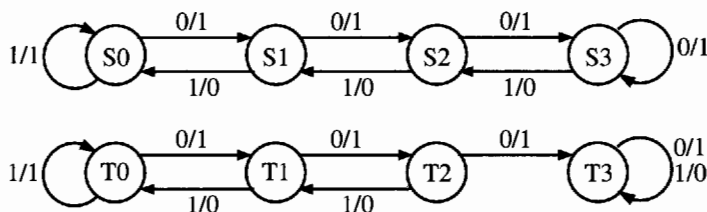
10.5

- (a) Redraw the code converter network of Figure 1-20 in the form of Figure 10-8 using dual-port flip-flops.
- (b) Determine a test sequence that will verify the first two rows of the transition table of Figure 1-18. Draw a timing diagram similar to Figure 10-9 for your test sequence.

10.6

- (a) Write VHDL code for a dual-port flip-flop.
- (b) Write VHDL code for your solution to Problem 10.5(a).
- (c) Write a test bench that applies the test sequence from Problem 10.5(b), and compare the resulting waveforms with your solution to Problem 10.5(b).

10.7 State graphs for two sequential machines are given below. The first graph represents a correctly functioning machine, and the second represents the same machine with a malfunction. Assuming that the two machines can be reset to their starting states (S0 and T0), determine the shortest input sequence that will distinguish the two machines.



10.8 Simulate the boundary scan tester of Figure 10-19 and verify that the results are as expected. Change the code to represent the case where the lower input to IC1 is shorted to ground, simulate again, and interpret the results.

10.9 Write VHDL code for the boundary scan cell of Figure 10-14. Rewrite the VHDL code of Figure 10-18 to use this boundary scan cell as a component in place of some of the behavioral code for the BSR. Use a generate statement to instantiate NCELLS copies of this component. Test your new code using the boundary scan tester example of Figure 10-19.

10.10

- (a) Draw a schematic diagram for an LFSR with $n = 5$ that generates a maximum length sequence.
- (b) Add logic so that 00000 is included in the state sequence.
- (c) Determine the actual state sequence.

10.11

- (a) Write VHDL for an 8-bit MISR that is similar to Figure 10-25.
- (b) Design a self-test circuit, similar to Figure 10-22, for a 6116 static RAM. The write-data generator should store data in the following sequence: 00000000, 10000000, 11000000, ..., 11111111, 01111111, 00111111, ..., 00000000.
- (c) Write VHDL code to test your design. Simulate the system for at least one example with no errors, 1 error, 2 errors, and 3 errors.